PHYS30762 Object Oriented Programming in C++

Assignment 5 Report

Kuba Kane
11019699

Department of Physics and Astronomy
The University of Manchester

April 2026

Contents

1 Program summary **1**
1.1 Electron class 1
1.2 Muon class 1
1.3 Tau class 2
1.4 Neutrino class 2
1.5 Main cpp file 2

2 References **3**

Note to markers: For version history please see the README file for different branches on the Github repository for assignment 5, as this contains planning and content for this report.

1. Program summary

This program is based around a `Lepton` class, which is an abstract base class from which `Electron`, `Muon`, `Tau`, and `Neutrino` are derived classes. The `Lepton` class contains a unique pointer to a `FourMomentum` object, whose class has energy, x momentum, y momentum, z momentum, and rest mass, as data members (all measured in MeV). The `Lepton` class also contains a rest mass and relative charge, as well as a boolean expression which if true represents an antilepton by flipping the charge data member. The `Lepton` class contains 2 virtual member functions which makes it abstract (along with its virtual destructor). The first is the `info()` function which is used to print out additional properties of derived classes, and `has_decayed()` which is set true if one of the derived leptons have decayed. `Lepton` contains `base_info()` to print out the properties it carries (four momentum, rest mass, and relative charge).

The derived classes inheret all these properties, but also have their own data members and member functions. These additional properties are printed by overriding the `info()` function (using `base_info()` to print the properties from `Lepton`), and are used to determine the outcome of `has_decayed()`. Figure X is a UML diagram which shows the heirarcy of the derived classes, and their properties. All of the derived classes use `static constexpr` to store their mass and charge as constants which cannot be modified during compilation or run time, which are printed via `base_info()`.

1.1. Electron class

This class introduces an interaction with a detector, primarily through calculating the shower depth in a tungsten (W) detector via the member function `shower_depth_in_tungsten()`. This function calculates the shower depth X using the equation

$$X = X_0 \frac{ln(E_0/E_c)}{ln2}, \tag{1}$$

where X_0 is the radiation length, E_c is the critical energy, and E_0 is the initial energy of the electron (from its `FourMomentum`). The first two of these are set within the member function, matching the values for tungsten. E_0 is accessed via the pointer to the `FourMomentum` from the Lepton base class. `Electron` also has the function `shower_contained(const double detector_length_in)` which, given a detector length, determines if the shower is contained within the detector.

1.2. Muon class

The `Muon` class introduces the data member `time_in_detector()` which, if greater than the constant `MUON_MEAN_LIFETIME` stored as a private data member in the class, sets `has_decayed()` to be true. A `validate_time()` static function is used to check that the time entered by the user is not negative or greater than 1 second.

1.3. Tau class

The `Tau` class uses `time_in_detector()` and with `TAU_MEAN_LIFETIME` to handle the `has_decayed()` function, akin to the `Muon` class. Unlike the `Muon` class, however, it takes into account the different decay modes of taus. Using the fact that 64% of the time a taus' decay products are hadronic, 17.8% of the time they involve electrons, and 17.4% of the time they involve muons, a private member function `generate_decay_mode()` uses a basic Monte Carlo simulation to set the `decay_mode` data member to one of the three named values in the enumerator class defined for the decay modes. The getter `get_decay_mode` uses a switch to return the decay products as a string, differing for each named value in the enumerator class, and this is used in the overriden `info()` function.

1.4. Neutrino class

In the neutrino class, the idea of lifetime is discarded and `has_decayed` is always set to false due to the stability of neutrinos. An enumerator class `NeutrinoFlavour` is used with three named values corresponding to the three neutrino flavours. The neutrino flavour is set as a private data member which can be specified via the neutrino constructor or flavour setter. Since a neutrino cannot be flavourless, the default constructor also requires flavour as an input. The `info()` function prints the flavour using a switch returning a string dependent on the named value in the enumerator class, as well as the flavour generation, again dependent on the named value.

1.5. Main cpp file

Text here

2. References

The following sources were used in the research for this assignment.

[1] https://www.geeksforgeeks.org/cpp/pure-virtual-functions-and-abstract-classes/

[2] https://en.cppreference.com/w/cpp/language/rule_of_three.html

[3] https://www.geeksforgeeks.org/cpp/understanding-constexper-specifier-in-cpp/

[4] https://www.geeksforgeeks.org/cpp/sqrt-sqrtl-sqrtf-cpp/

[5] https://www.geeksforgeeks.org/cpp/enum-classes-in-c-and-their-advantage-over-enum-datatype/

[6] https://www.geeksforgeeks.org/cpp/range-based-loop-c/

[7] https://en.wikipedia.org/wiki/Particle_decay

[8] https://physics.stackexchange.com/questions/282245/muon-decay-mode

[9] https://en.wikipedia.org/wiki/Muon

[10] https://en.wikipedia.org/wiki/Tau_(particle)

[11] https://tikz.net/sm_decay_matrix/

[12] https://www.geeksforgeeks.org/cpp/stduniform_real_-distribution-class-in-c-with-examples/

[13] https://www.w3schools.com/cpp/cpp_switch.asp

[14] https://www.geeksforgeeks.org/cpp/can-constructor-private-cpp/

[15] https://www.w3schools.com/cpp/cpp_constructors_overloading.asp

[16] https://www.geeksforgeeks.org/system-design/factory-method-pattern-c-design-patterns/

[17] https://pdg.lbl.gov/2020/AtomicNuclearProperties/HTML/tungsten_W.html

[18] https://en.wikipedia.org/wiki/Particle_shower

[19] https://indico.capa.unizar.es/event/23/sessions/48/attachments/287/393/COST-TS-Belgrade-Course5-Tutorial.pdf